

Detección de movimiento *en video*

2019-I

Carlos Alejandro Trujillo Anaya – carlostrujillo@itm.edu.co

Oficina M212

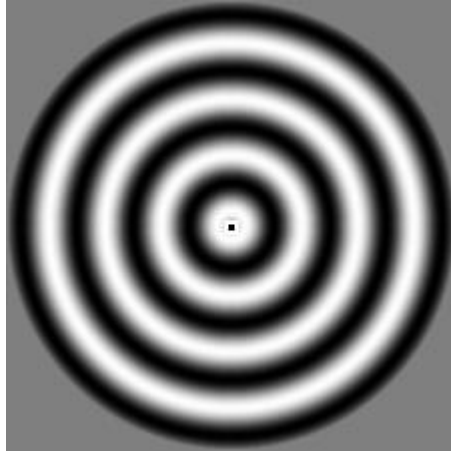
Departamento de electrónica y telecomunicaciones

Asignatura perteneciente a Ingeniería electrónica

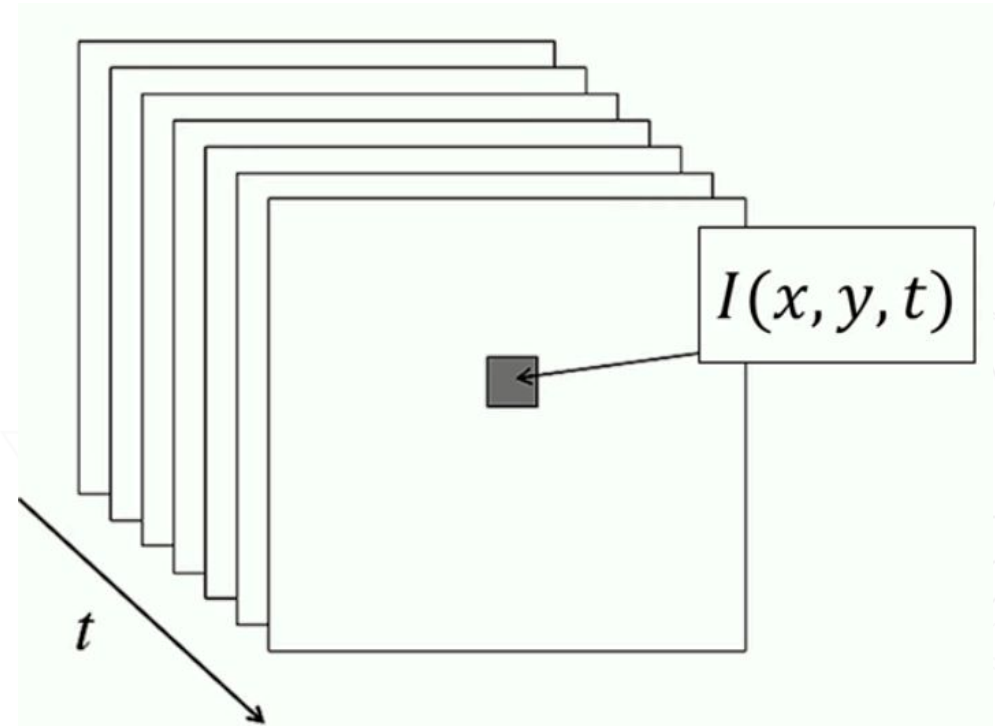
Viernes 18-20

Aula O2

En qué consiste el movimiento en un video?



<http://www.cns.nyu.edu/~david/courses/perception/lecturenotes/motion/motion.html>



En el video, la función deja de depender únicamente del espacio, para también depender del tiempo.

Aplicaciones de la detección de movimiento

- Estabilización de imágenes automática (en cámaras digitales).



<https://www.theverge.com/2014/10/13/6968699/hyperlapse-is-really-good-at-stabilized-video>

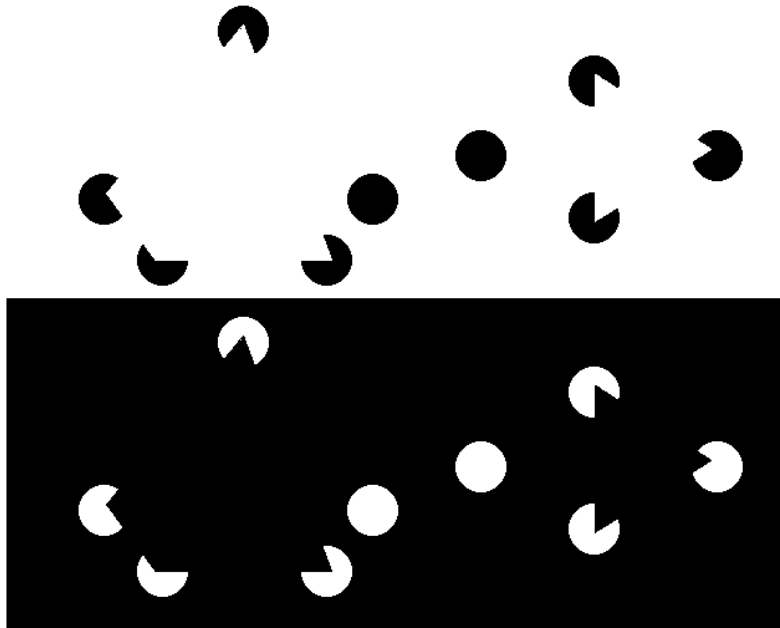
Aplicaciones de la detección de movimiento

- Substracción de fondo



<https://classroom.udacity.com/courses/ud810/>

Percepción de movimiento



https://www.settheory.com/Glass_paper/Kanizsa_observations.html

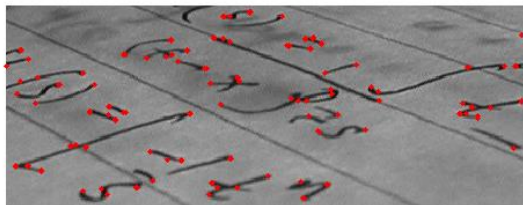
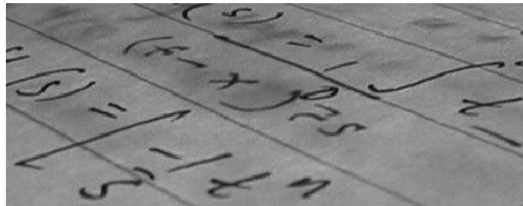


Cambios espaciales entre los *frames* de un video nos llevan a construir la **noción de movimiento** en el mismo.

Técnicas de estimación de movimiento *en video*

Métodos basados en características:

- Detectar características en una imagen (práctica anterior) y seguirlas en las siguientes.
- Ideal para detectar ***movimientos fuertes***.



https://en.wikipedia.org/wiki/Corner_detection

Técnicas de estimación de movimiento *en video*

Métodos directos (estimaciones de densidad):

- Detectar el movimiento de cada pixel a partir de la ***variación de la intensidad*** en el espacio y en el tiempo.
- Podemos pensar en un ***flujo de movimiento***.
- Ideal para detectar movimientos pequeños.



https://docs.opencv.org/3.4/d7/d8b/tutorial_py_lucas_kanade.html

Flujo óptico

Es el movimiento aparente de objetos o superficies.

$$E_{\text{SSD-OF}}(\{u_i\}) = \sum_i [I_1(x_i + u_i) - I_0(x_i)]^2$$

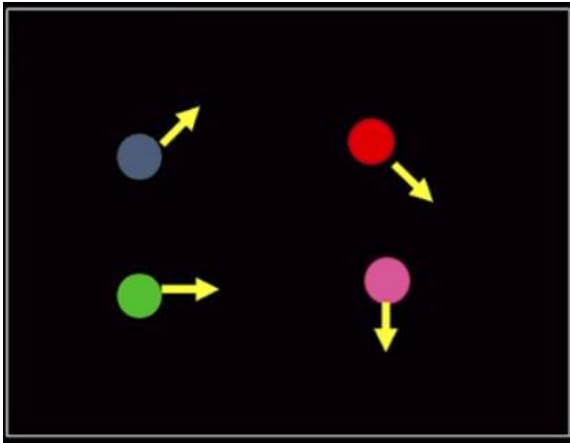


Objetivo: Se desea minimizar el brillo o la diferencia de color entre los píxeles correspondientes sumados sobre la toda la imagen.

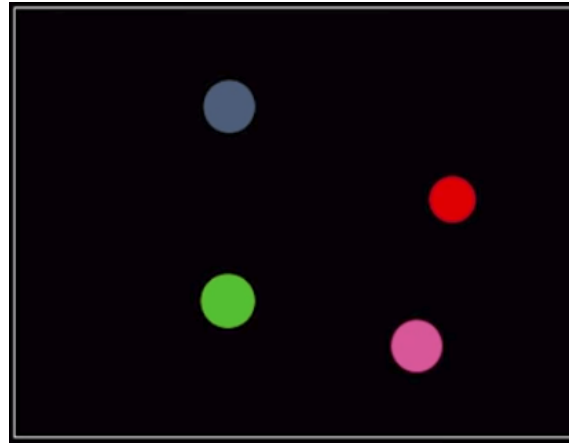
Szeliski, Computer vision: Algorithms and applications. Chapter 8.

Flujo óptico

Problema: Calcular las flechas amarillas las cuales determinan el movimiento de cada pixel entre *frames* consecutivos.



$I(x, y, t)$

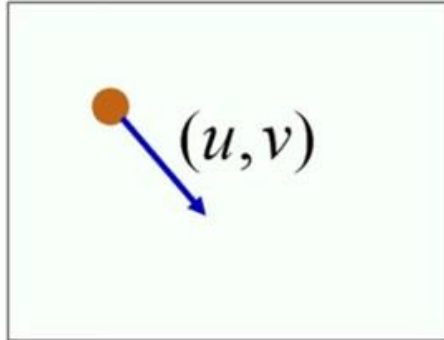


$I(x, y, t + 1)$

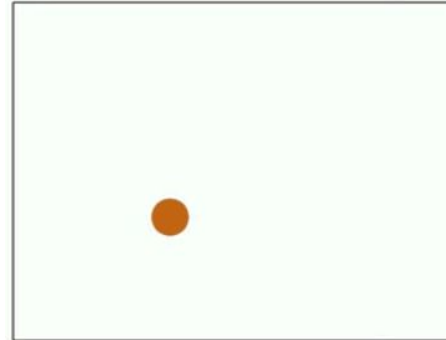
Color?
Vecindario?

Flujo óptico

Supondremos que u y v son pequeños (del orden de un par de píxeles máximo).



$I_0(x, y, t)$



$I_1(x, y, t+1)$

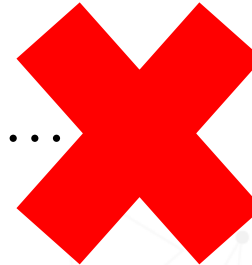
Restricción de constancia del brillo

$$0 = I_1(x + u, y + v, t + 1) - I_0(x, y, t)$$

Flujo óptico

Expandiendo en series de Taylor a I_1 (componente espacial):

$$I_1(x + u, y + v) = I_0(x, y) + \frac{\partial I_1}{\partial x} u + \frac{\partial I_1}{\partial y} v + \dots$$



Obtenemos una ***aproximación*** que nos permitirá resolver el problema!

Flujo óptico

De esta manera podemos reescribir la restricción

$$0 = I_1(x + u, y + v, t + 1) - I_0(x, y, t)$$

$$0 \approx I_0(x, y) + \frac{\partial I_1}{\partial x} u + \frac{\partial I_1}{\partial y} v - I_0(x, y, t)$$

$$0 \approx I_0(x, y) - I_0(x, y, t) + \frac{\partial I_1}{\partial x} u + \frac{\partial I_1}{\partial y} v$$

Obtenemos una **aproximación** que nos permite resolver el problema!

Flujo óptico

De esta manera podemos reescribir la restricción

$$0 \approx \frac{\partial I_0}{\partial t} + \nabla I_1 \cdot (u, v)$$

Donde (u, v) es el vector de ***dirección de movimiento!***

En el límite t tiende a ***cero***, esta expresión es la ***ecuación*** de restricción de constancia de brillo (o color).

Tenemos un sistema de ecuaciones ***subestimado!***

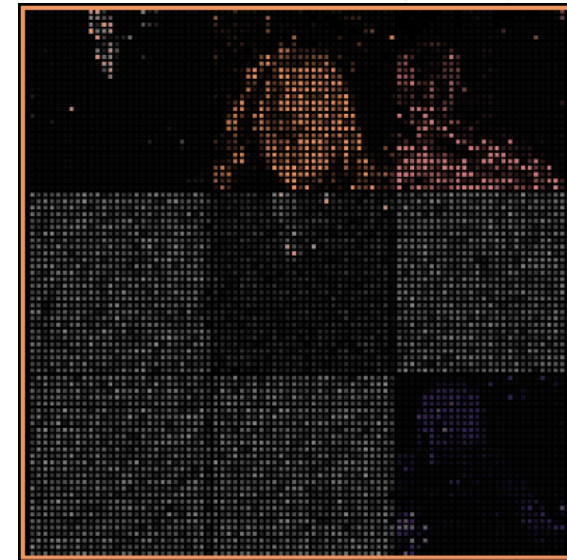
$$e_c = \iint_{image} (I_x u + I_y v + I_t)^2 dx dy$$

Flujo óptico

Necesitamos más restricciones para solucionar el sistema de ecuaciones.

Restricción de suavidad

$$e_s = \iint_{image} (u_x^2 + u_y^2) + (v_x^2 + v_y^2) \, dx \, dy$$



<http://cmuems.com/2018/60212f/category/04/04-telematic/>

Flujo óptico (*Implementaciones*)

Lucas-Kanade Optical Flow in OpenCV

Se utiliza una ventana de 3x3 alrededor de cada punto, se resuelve para el gradiente de I (o f en openCV) y para la derivada de I respecto al tiempo.

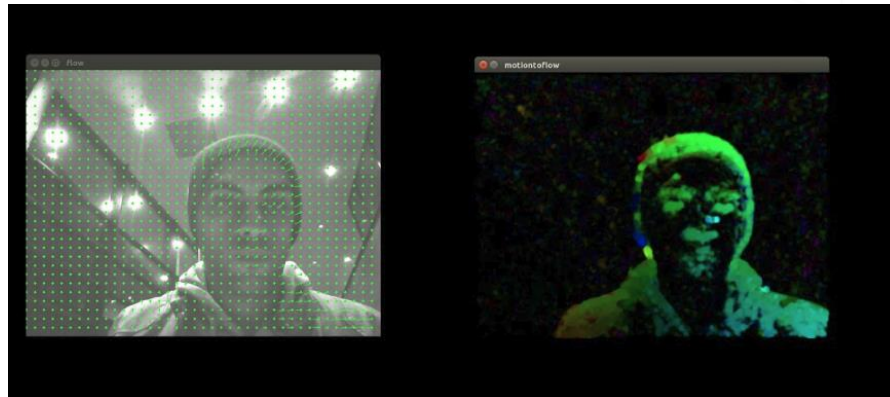
$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} \sum_i f_{x_i}^2 & \sum_i f_{x_i} f_{y_i} \\ \sum_i f_{x_i} f_{y_i} & \sum_i f_{y_i}^2 \end{bmatrix}^{-1} \begin{bmatrix} -\sum_i f_{x_i} f_{t_i} \\ -\sum_i f_{y_i} f_{t_i} \end{bmatrix}$$

Se utilizan **pirámides** para detectar **movimientos fuertes**!

Flujo óptico (*Implementaciones*)

Gunner Farneback's algorithm Optical Flow in OpenCV

En vez de realizar la búsqueda para características particulares, se implementa una búsqueda sobre todos los puntos (densidad) en la imagen.



<https://www.youtube.com/watch?v=IOoYCu939kl>

Se obtienen **mapas de magnitud y dirección** de movimiento en toda la imagen!

https://docs.opencv.org/3.4/d7/d8b/tutorial_py_lucas_kanade.html

Referencias

Szeliski, ***Computer vision: Algorithms and applications***. Chapter 8.

https://docs.opencv.org/3.4/d7/d8b/tutorial_py_lucas_kanade.html